
TypeScript for JavaScript Developers

The slide features a light blue background with decorative elements. At the top, there are two horizontal lines: a thick teal line followed by a thin light blue line. At the bottom, there are two horizontal lines: a thin light blue line followed by a thick teal line. Two short, thick olive-green horizontal bars are positioned on the left and right sides of the slide, below the main title.

Introduction

TypeScript is **JavaScript with syntax for types**

It checks a program for errors before execution, and does so based on the kinds of values, it's a **static type checker**

It is a **strongly typed programming language** that **builds on JavaScript**

TypeScript code converts to JavaScript, which runs anywhere JavaScript runs: In a **browser**, on **Node.js** or **Deno** and in your apps

Types by Inference

TypeScript knows the JavaScript language and will generate types for you in many cases

```
let helloWorld = "Hello world";
```

```
let helloWorld: string
```

Defining Types

An object with an **inferred type** which includes

`name: string` and `id: number`

```
const user = {  
  name: "Hayes",  
  id: 0,  
};
```

Defining Types

Explicitly **defining** object shape

```
interface User {  
  name: string;  
  id: number;  
}
```

```
const user: User = {  
  name: "Hayes",  
  id: 0,  
};
```

Defining Types

The compiler will not allow you to use a properties not defined in the shape

```
const user: User = {  
  username: "Hayes",  
}
```

Type '{ username: string; id: number; }' is not assignable to type 'User'.

Object literal may only specify known properties, and 'username' does not exist in type 'User'.

```
  id: 0,  
};
```

Defining Types

Since JavaScript supports classes and object-oriented programming, so does TypeScript

```
class UserAccount {  
    name: string;  
    id: number;  
  
    constructor(name: string, id: number) {  
        this.name = name;  
        this.id = id;  
    }  
}
```

Defining Types

You can use an interface declaration with classes

```
interface User {  
  name: string;  
  id: number;  
}
```

```
class UserAccount {  
  name: string;  
  id: number;  
  
  constructor(name: string, id: number) {  
    this.name = name;  
    this.id = id;  
  }  
}
```

```
const user: User = new UserAccount("Murphy", 1);
```

Defining Types

You can **use interfaces** to **specify type of parameters** and **return values** to functions

```
function getAdminUser(): User {  
    //...  
}  
  
function deleteUser(user: User) {  
    // ...  
}
```

Defining Types

JavaScript types

`boolean`

`bigint`

`null`

`number`

`string`

`symbol`

`undefined`

Defining Types

Types added in TypeScript

any - allow anything

unknown - ensure someone using this type declares what the type is

never - it's not possible that this type could happen

void - a function which returns **undefined** or has **no return value**

Composing Types

With TypeScript, you can create **complex types** by **combining simple ones**

There are two popular ways to create complex types:

- with **unions**

- with **generics**

Composing Types

Unions

With a union, you can declare that a type could be one of many types

Examples:

```
type MyBool = true | false;
```

Composing Types

Unions

With a union, you can declare that a type could be one of many types

Examples:

```
type WindowStates = "open" | "closed" | "minimized";  
type LockStates = "locked" | "unlocked";  
type PositiveOddNumbersUnderTen = 1 | 3 | 5 | 7 | 9;
```

Composing Types

Unions

With a union, you can declare that a type could be one of many types

Examples:

```
function getLength(obj: string | string[]) {  
    return obj.length;  
}
```

Composing Types

Generics

Generics provide **variables to types**

Examples:

```
type StringArray = Array<string>;  
type NumberArray = Array<number>;  
type ObjectWithNameArray = Array<{ name: string }>;
```

Composing Types

Generics

Generics provide **variables to types**

Examples:

```
interface Backpack<Type> {  
    add: (obj: Type) => void;  
    get: () => Type;  
}
```

```
declare const backpack: Backpack<string>;
```

Composing Types

Generics

Generics provide **variables to types**

Examples:

```
declare const backpack: Backpack<string>;
```

```
// Since the backpack variable is a string, you can't pass a number  
backpack.add(23);
```

Argument of type 'number' is not assignable to parameter of type 'string'.

Structural Type System

In TypeScript **type checking** focuses on **the shape that values** have

this is called “duck typing” or “**structural typing**”

In a structural type system, **if two objects have the same shape, they are considered to be of the same type**

Structural Type System

```
interface Point {  
  x: number;  
  y: number;  
}  
  
function logPoint(p: Point) {  
  console.log(`${p.x}, ${p.y}`);  
}
```

```
// logs "12, 26"  
const point = { x: 12, y: 26 };  
logPoint(point);
```

Structural Type System

The **shape-matching** only requires a subset of the object's fields to match

```
const point3 = { x: 12, y: 26, z: 89 };  
logPoint(point3); // logs "12, 26"  
  
const rect = { x: 33, y: 3, width: 30, height: 80 };  
logPoint(rect); // logs "33, 3"
```

Structural Type System

The **shape-matching** only requires a subset of the object's fields to match

```
const color = { hex: "#187ABF" };  
logPoint(color);
```

Argument of type '{ hex: string; }' is not assignable to parameter of type 'Point'.

Type '{ hex: string; }' is missing the following properties from type 'Point': x, y

Structural Type System

There is **no difference** between how classes and objects conform to shapes

```
class VirtualPoint {  
  x: number;  
  y: number;  
  
  constructor(x: number, y: number) {  
    this.x = x;  
    this.y = y;  
  }  
}  
  
const newVPoint = new VirtualPoint(13, 56);  
logPoint(newVPoint); // logs "13, 56"
```

Reference

<https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes.html>

<https://mariusschulz.com/blog/the-unknown-type-in-typescript>

<https://www.typescriptlang.org/docs/handbook/intro.html>

<https://www.typescriptlang.org/play#show-examples>